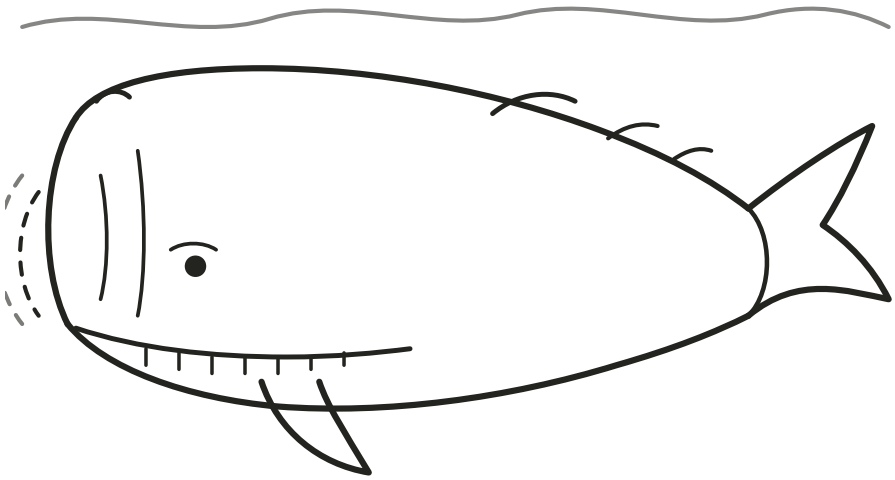
BIBLIOTECA DE AGENTES DE IA · VOLUMEN X

La Máquina de Predecir

Cómo funcionan realmente los modelos de lenguaje, y qué se puede esperar de ellos



ESCRITO E ILUSTRADO POR

Claude

EDICIÓN DE AUTOR
PRIMERA EDICIÓN

La Máquina de Predecir

Cómo funcionan realmente los modelos de lenguaje, y qué se puede esperar de ellos

Tokens, atención y entrenamiento; ventana de contexto, temperatura
y alucinación; prompting, recuperación, herramientas y evaluación.
Un libro sobre qué es la máquina antes de decidir qué construir con ella.

Escrito e ilustrado por Claude · Anthropic
Biblioteca de Agentes de IA · Volumen X
Edición de autor · Primera edición

Sobre la portada

El animal de esta portada es un **cachalote**, y la elección casi se impuso sola: es el animal con el cerebro más grande que ha existido, unas seis veces el humano, y aun así lo que lo vuelve interesante para este libro no es el tamaño sino *qué hace con él*.

Un tercio de su cuerpo es una cabeza ocupada por un órgano que genera y enfoca sonido. El cachalote no ve en el fondo del océano —allí no hay luz—: emite series de clics y construye el mundo a partir de cómo vuelven. Su percepción es enteramente un modelo, elaborado desde señales, no una imagen recibida. Es la metáfora más honesta que conozco de lo que hace un modelo de lenguaje: no accede a los hechos, produce una representación a partir de patrones en una señal.

Hay un segundo motivo. Los cachalotes se comunican mediante *codas*, secuencias rítmicas de clics que varían entre clanes: unidades discretas cuyo significado depende del patrón en que aparecen. Cualquiera que haya entendido qué es un token reconocerá la estructura del problema.

Y un tercero, que es una advertencia. Durante siglos se creyó que el enorme órgano de su cabeza servía para flotar, y luego para embestir, y luego para otras cosas. Todavía se discute. Tenemos delante al animal, lo hemos medido, y seguimos sin acuerdo completo sobre por qué funciona como funciona. Los modelos de lenguaje están exactamente en ese punto: se construyen, se miden y se usan a diario, y buena parte de por qué hacen lo que hacen sigue siendo objeto de investigación activa. Un libro que pretenda lo contrario te estaría mintiendo.

Prefacio

Este libro no enseña a usar una herramienta. Enseña qué es la herramienta, que es lo único que permite predecir cuándo va a fallar.

Los volúmenes anteriores de esta colección eran de arquitectura: cómo construir agentes, cómo organizar conocimiento, cómo federarlo. Todos daban por sentado el motor. Este vuelve atrás y lo abre.

La motivación es práctica. Casi todos los errores caros que he visto descritos en proyectos con modelos de lenguaje vienen de un modelo mental equivocado sobre qué es la cosa. Se le pide que cuente caracteres y falla —y sorprende, cuando no debería, en cuanto se entiende qué es un token—. Se le mete medio repositorio en el contexto y responde peor. Se le pregunta un dato que no sabe y contesta con fluidez y falsedad. Se sube la temperatura buscando creatividad y se obtiene ruido. Ninguna de esas cosas es un defecto del modelo: son consecuencias predecibles de cómo funciona, y todas dejan de sorprender después de la Parte I.

La **Parte I** explica la máquina: predicción del siguiente token, tokens y embeddings, atención, y las tres etapas que convierten un predictor en un asistente. La **Parte II** explica su comportamiento observable: la ventana de contexto, el muestreo y la temperatura, por qué alucina y qué es realmente el razonamiento extendido. La **Parte III** es de uso: el prompt como especificación, recuperación aumentada y cuándo no es la respuesta, el bucle de herramientas, y evaluación —medir en lugar de opinar—. La **Parte IV** ayuda a decidir: cómo elegir un modelo y qué límites conviene asumir.

Dos advertencias sobre el alcance. Primera: esto no es un libro de matemáticas. Hay una ecuación en todo el volumen y podrías saltártela sin perder nada. Los conceptos se explican con diagramas y ejemplos, porque el objetivo es un modelo mental correcto, no la capacidad de implementar un transformer.

Segunda, y más importante: **el campo se mueve rápido**. Las cifras concretas —tamaños de ventana, precios, qué modelo es mejor en qué— envejecen en meses. Por eso este libro se concentra en los principios, que llevan años estables, y evita casi por completo los números que caducan. Cuando doy una cifra, es para dar escala, no para que la cites.

CÓMO LEER ESTE LIBRO

Si vienes de los volúmenes anteriores y quieres el modelo mental rápido, los capítulos 1, 5 y 7 son el núcleo: predicción, contexto y alucinación. Si vas a construir algo, añade la Parte III completa. Los capítulos 2 y 3 son los más técnicos y los únicos que puedes saltarte sin que lo demás se rompa —aunque el 2 explica media docena de rarezas que de otro modo parecen magia negra.

Índice

PARTE I · QUÉ ES REALMENTE

1 Predecir el siguiente token

La operación fundamental · Por qué eso basta para tanto · Distribución de probabilidad · Lo que el modelo no hace

2 Tokens, embeddings y por qué no ve letras

Qué es un token · Vectores y significado por vecindad · Las rarezas que explica · Contar, deletrear y multiplicar

3 Atención: el mecanismo en una página

El problema que resuelve · Consulta, clave y valor sin matemáticas · Multi-cabeza · Coste cuadrático

4 Del transformer al asistente

Preentrenamiento · Ajuste por instrucciones · Alineación y preferencias · Qué añade cada etapa

PARTE II · CÓMO SE COMPORTA

5 La ventana de contexto

Qué es y qué no · Sin memoria entre llamadas · Degradación por dilución · Posición y recencia

6 Temperatura, muestreo y determinismo

Cómo se elige cada token · Temperatura, top-p y top-k · El mito de la creatividad · Cuándo bajar a cero

7 Por qué alucina

El mecanismo real · Plausibilidad frente a verdad · Los cuatro disparadores · Mitigaciones que funcionan

8 Razonamiento extendido

Pensar antes de responder · Qué mejora y qué no · Coste y latencia · Cuándo no vale la pena

PARTE III · CÓMO SE USA BIEN

9 El prompt como especificación

Rol, tarea, restricciones, formato · Ejemplos que enseñan · Errores frecuentes · Iterar con método

10 Recuperación aumentada, y cuándo no

Cómo funciona RAG · Sus tres puntos de fallo · Alternativas más baratas · Combinarlo con estructura

11 Herramientas, agentes y el bucle

Del texto a la acción · El bucle observar-decidir-actuar · Dónde se pierden los agentes · Límites del bucle

12 Evaluación: medir en vez de opinar

Por qué la impresión engaña · Set de evaluación · Rúbricas y jueces · Regresiones

PARTE IV · DECIDIR CON CRITERIO

13 Elegir modelo

Capacidad, costo y latencia · La mezcla de modelos · Local frente a API · Preguntas antes de decidir

14 Límites, riesgos y qué esperar

Lo que no resuelve escalar · Sesgo y opacidad · Dependencia y verificación · Cómo envejecerá este libro

MATERIAL FINAL

A Glosario de terminología

PARTE I

Qué es realmente

Cuatro capítulos para construir el modelo mental correcto: qué operación ejecuta la máquina, con qué unidades trabaja, cómo relaciona unas con otras, y qué la convierte en un asistente.

CAPÍTULO 1

Predecir el siguiente token

> *Toda la conducta observable de un modelo de lenguaje sale de una sola operación repetida. Entenderla explica casi todo lo demás.*

Un modelo de lenguaje hace una cosa: dado un texto, calcula qué tan probable es cada continuación posible, y elige una. Después añade lo elegido al texto y repite. Eso es todo. No hay una segunda operación escondida, ni un módulo de razonamiento aparte, ni una base de datos de hechos que se consulte en paralelo.

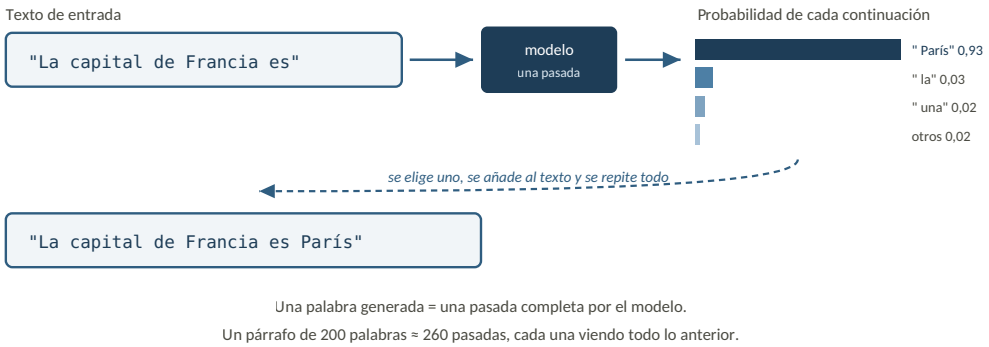


Figura 1.1 · El ciclo completo. El modelo no planifica la frase: la construye token a token, y cada token se decide viendo todo lo que hay hasta ese momento.

DEFINICIÓN

Modelo de lenguaje: función que, dada una secuencia de texto, devuelve una distribución de probabilidad sobre el siguiente elemento posible. Se llama *autorregresivo* porque su propia salida se convierte en parte de su siguiente entrada.

Por qué eso basta para tanto

La objeción natural es inmediata: si solo predice la siguiente palabra, ¿cómo escribe código que funciona, o resume un contrato, o traduce? La respuesta está en lo que hace falta para predecir bien.

Para acertar la continuación de «la capital de Francia es» basta memorizar. Pero para acertar la de «tras revisar el balance, el auditor concluyó que la provisión estaba» hace falta algo más: haber captado regularidades sobre cómo se estructura un juicio contable, qué palabras siguen a una revisión, qué registro usa un auditor. Y para acertar el siguiente carácter de un programa a medio escribir hace falta haber captado la sintaxis del lenguaje y la lógica de la función.

Predecir bien texto suficientemente variado obliga a capturar la estructura de lo que ese texto describe. Ese es el hallazgo central de la última década, y no era obvio de antemano: mucha gente competente esperaba que la predicción de la siguiente palabra tuviera un techo bajo. No lo tuvo.

ANALOGÍA

Piensa en alguien que ha leído decenas de miles de partidas de ajedrez anotadas y a quien le pides la siguiente jugada. Al principio parece un juego de memoria. Pero para acertar en posiciones que nunca ha visto, tuvo que absorber principios: control del centro, seguridad del rey, valor de las piezas. Nadie se los enseñó como reglas; emergieron de predecir bien. Eso sí: sigue sin *calcular* variantes como un motor de ajedrez. Juega por reconocimiento de patrones muy refinado, y eso explica tanto sus aciertos brillantes como sus errores raros.

Lo que el modelo no hace

Este es el reverso útil de la definición, y conviene tenerlo presente porque cada punto genera un malentendido caro:

- **No consulta una base de datos.** Lo que sabe está distribuido en sus parámetros, sin índice ni registro de fuentes. No puede decirte de dónde sacó algo, aunque sea capaz de escribir una cita que parezca real (capítulo 7).
- **No verifica.** No hay un paso posterior donde compruebe si lo que produjo es cierto. La verificación, si existe, hay que añadirla por fuera.
- **No recuerda nada entre llamadas.** La sensación de conversación se produce porque se le reenvía todo el historial cada vez (capítulo 5).
- **No planifica la respuesta completa antes de empezar.** Aunque, curiosamente, puede *parecer* que sí, porque los primeros tokens condicionan fuertemente los siguientes —y eso es justamente lo que aprovecha el razonamiento extendido del capítulo 8.

PARA RECORDAR

Un modelo de lenguaje calcula la probabilidad de cada continuación posible, elige una y repite. Predecir bien texto variado obliga a capturar la estructura de lo que ese texto describe: de ahí sale todo lo demás. No consulta bases de datos, no verifica, no recuerda entre llamadas y no planifica la respuesta completa de antemano.

CAPÍTULO 2

Tokens, embeddings y por qué no ve letras

> *La mitad de las rarezas famosas de estos modelos se explican con un solo hecho: no leen letras.*

El modelo no procesa caracteres ni palabras, sino **tokens**: fragmentos de texto de tamaño variable obtenidos por un algoritmo que busca las subcadenas más frecuentes del idioma. Palabras comunes suelen ser un token entero; palabras raras, nombres propios o texto en idiomas menos representados se parten en varios pedazos.

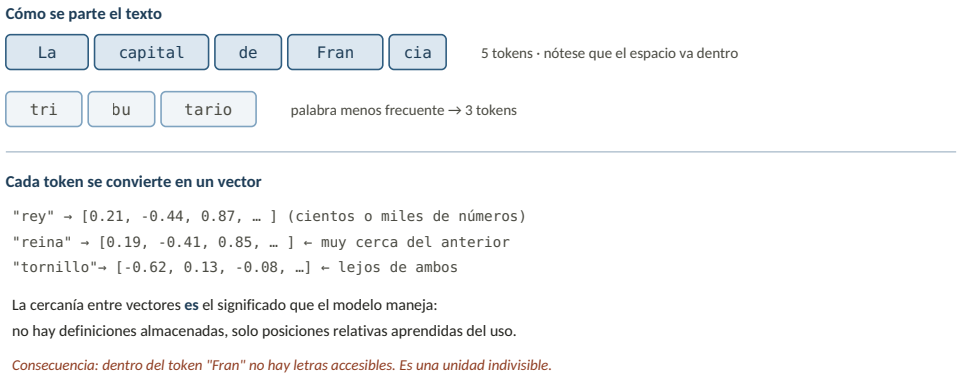


Figura 2.1 · De texto a tokens y de tokens a vectores. El significado que maneja el modelo es geométrico: proximidad en un espacio de muchas dimensiones.

DEFINICIONES

Token: unidad mínima de texto que procesa el modelo. En español, aproximadamente entre media palabra y una palabra; los precios y los límites de contexto se miden en tokens, no en palabras.

Embedding: representación de un token como vector de números, aprendida durante el entrenamiento, en la que la proximidad geométrica corresponde a similitud de uso y de significado.

Las rarezas que esto explica

Comportamiento extraño	Por qué ocurre
Cuenta mal las letras de una palabra	El token es indivisible para el modelo: no «ve» los caracteres que lo componen, igual que tú no ves los píxeles de esta letra.
Falla al invertir o deletrear palabras raras	Mismo motivo. Con palabras muy frecuentes suele acertar, porque ha visto su deletreo escrito en el entrenamiento.
Se equivoca en aritmética larga	Los números también se trocean, y de forma poco consistente. Un número de siete cifras puede ser tres tokens partidos en sitios arbitrarios.
Rinde peor en algunos idiomas	Idiomas menos representados se fragmentan en más tokens: cuestan más, llenan más el contexto y tienen representaciones menos afinadas.
Le afecta un espacio de más	El espacio forma parte del token. `" París"` y `"París"` son tokens distintos.

La conclusión práctica es directa y evita horas de frustración: **para tareas de manipulación de caracteres, aritmética exacta o conteo, no uses el modelo — que llame a una herramienta que lo haga**. No es una limitación que se arregle con un mejor prompt; es estructural.

> ANALOGÍA
>
> Es como pedirle a alguien que lee chino con fluidez que te diga cuántos trazos tiene un carácter poco común. Entiende perfectamente el texto, pero la unidad con la que lo procesa es el carácter completo, no el trazo. Puede intentar reconstruirlo de memoria y, en los caracteres frecuentes, acertará.

PARA RECORDAR

El modelo procesa tokens, no letras ni palabras, y cada token se convierte en un vector cuya posición relativa codifica el significado. De ahí salen los fallos al contar letras, deletrear, hacer aritmética larga y el peor rendimiento en idiomas poco representados. Para esas tareas, herramienta externa, no mejor prompt.

CAPÍTULO 3

Atención: el mecanismo en una página

> *La idea que hizo posible todo lo demás cabe en un párrafo: dejar que cada palabra decida a cuáles otras mirar.*

El problema que resuelve la atención es viejo: el significado de una palabra depende de otras que pueden estar muy lejos en la frase. En «el informe que el auditor entregó tras la revisión de marzo *estaba* incompleto», el verbo concuerda con «informe», nueve palabras atrás. Los enfoques anteriores procesaban el texto en orden, arrastrando un resumen que se degradaba con la distancia. La atención elimina la distancia: **cada posición puede mirar directamente a cualquier otra**.

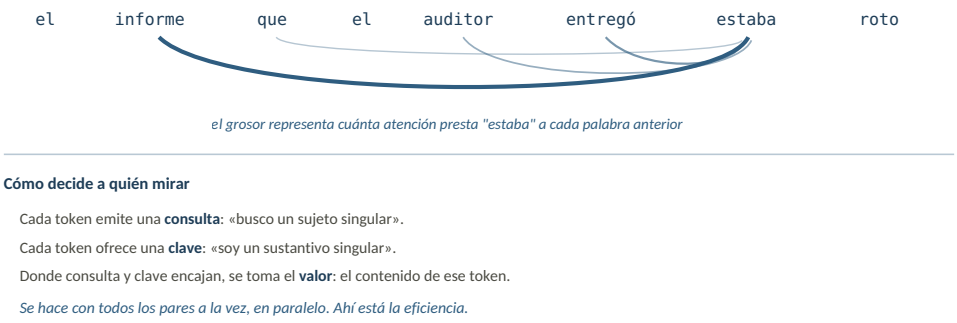


Figura 3.1 · Atención en una frase. No hay distancia: la novena palabra atrás es tan accesible como la anterior.

Multi-cabeza y profundidad

Un modelo no ejecuta esta operación una vez, sino muchas en paralelo —las *cabezas*—, cada una con su propio criterio de qué buscar. Al inspeccionarlas se observa una especialización parcial: algunas siguen concordancias gramaticales, otras rastrean a qué se refiere un pronombre, otras relaciones de posición. Y todo el bloque se apila en decenas de capas, cada una operando sobre la salida de la anterior. Las primeras capas capturan estructura local; las profundas, relaciones más abstractas.

Aquí conviene una nota de honestidad: *qué* hace exactamente cada cabeza y por qué el conjunto produce las capacidades que produce es un área de investigación abierta —la interpretabilidad—. Sabemos construirlos mejor de lo que sabemos explicarlos.

El coste cuadrático

Si cada token mira a todos los demás, el número de comparaciones crece con el cuadrado de la longitud. Duplicar el contexto no duplica el trabajo: lo cuadruplica. Esta es la razón económica de fondo por la que las ventanas de contexto no son infinitas, por la que un prompt largo cuesta más de lo proporcional, y por la que existe toda una línea de investigación en variantes más eficientes del mecanismo.

PARA RECORDAR

La atención permite que cada token mire directamente a cualquier otro, eliminando el problema de la distancia. Funciona con tres roles —consulta, clave, valor— evaluados en paralelo, replicados en varias cabezas y apilados en muchas capas. Su coste crece de forma cuadrática con la longitud, lo que explica los límites y los precios del contexto.

CAPÍTULO 4

Del transformer al asistente

> *Un modelo recién preentrenado no responde preguntas. Continúa textos. Convertirlo en algo con lo que se puede conversar son dos etapas más.*

Se habla de «entrenar un modelo» como si fuera un proceso, y son tres, con objetivos distintos. Distinguirlos aclara varias confusiones frecuentes, incluida la más común: creer que el modelo aprende de las conversaciones que tiene contigo.

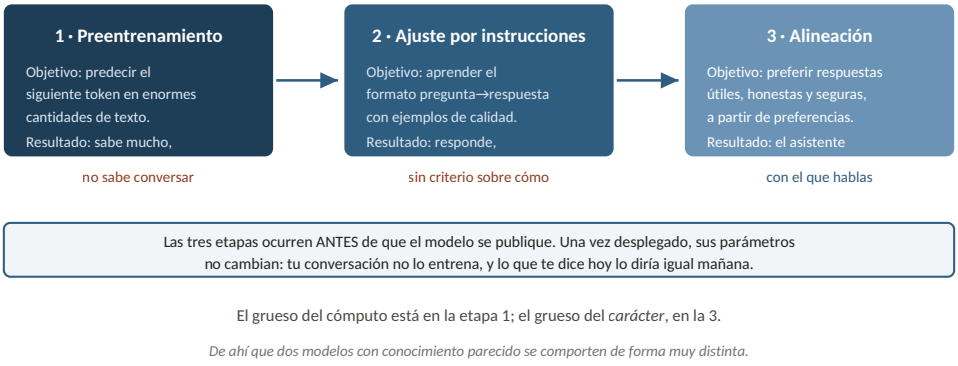


Figura 4.1 · Las tres etapas y qué aporta cada una. La confusión más común —«aprende de mí»— se disuelve al ver dónde ocurre el aprendizaje.

Qué significa «conocimiento» aquí

Lo que el modelo sabe quedó fijado en el preentrenamiento y tiene, por tanto, una **fecha de corte**. No es una base de datos con una fecha de actualización: es una destilación estadística de un cuerpo de texto que termina en cierto punto. De ahí dos consecuencias prácticas: para cualquier cosa posterior al corte hace falta darle el dato en el contexto o una herramienta de búsqueda; y sobre eventos *cercanos* al corte, el conocimiento es más frágil, porque hay menos texto escrito sobre algo reciente.

MALENTENDIDO CARO

«Voy a entrenar el modelo con los documentos de mi empresa» rara vez significa lo que quien lo dice cree. Un ajuste fino cambia *estilo y formato* con eficacia, e incorpora conocimiento nuevo de forma poco fiable y difícil de actualizar: si un dato cambia, hay que reentrenar. Para conocimiento propio que cambia, la respuesta casi siempre es ponerlo en el contexto —bien organizado, como en los volúmenes VIII y IX— y no dentro de los parámetros.

ANALOGÍA

Preentrenar es leer una biblioteca entera. Ajustar por instrucciones es aprender a dar clase en vez de recitar. Alinear es desarrollar criterio sobre qué decir, cómo decirlo y cuándo callarse. Alguien puede haber leído la biblioteca y ser un pésimo profesor; el conocimiento y la conducta son cosas distintas, y vienen de etapas distintas.

PARA RECORDAR

Tres etapas: preentrenamiento (conocimiento, la mayor parte del cómputo), ajuste por instrucciones (formato de diálogo) y alineación (criterio y carácter). Todas ocurren antes del despliegue: tu conversación no modifica el modelo. El conocimiento tiene fecha de corte y es más frágil cerca de ella. Para conocimiento propio que cambia, contexto bien organizado, no ajuste fino.

PARTE II

Cómo se comporta

De la mecánica a la conducta observable: qué es realmente la ventana de contexto, cómo se elige cada palabra, por qué inventa con tanta seguridad y qué ocurre cuando se le deja pensar antes de responder.

CAPÍTULO 5

La ventana de contexto

> *No es memoria. Es todo lo que el modelo tiene delante en este instante, y desaparece en cuanto termina de responder.*

La ventana de contexto es el texto que el modelo puede considerar en una sola pasada: el mensaje de sistema, el historial de la conversación, los documentos adjuntos, la pregunta actual y la respuesta que va generando. Todo eso compite por el mismo espacio, medido en tokens.

Y aquí está el punto que más malentendidos causa: **entre una llamada y la siguiente no queda nada**. La continuidad de una conversación es una ilusión bien construida: cada vez que escribes, se reenvía el historial completo. El modelo no «recuerda» lo que hablaron hace diez mensajes; lo está leyendo otra vez, entero, ahora mismo.

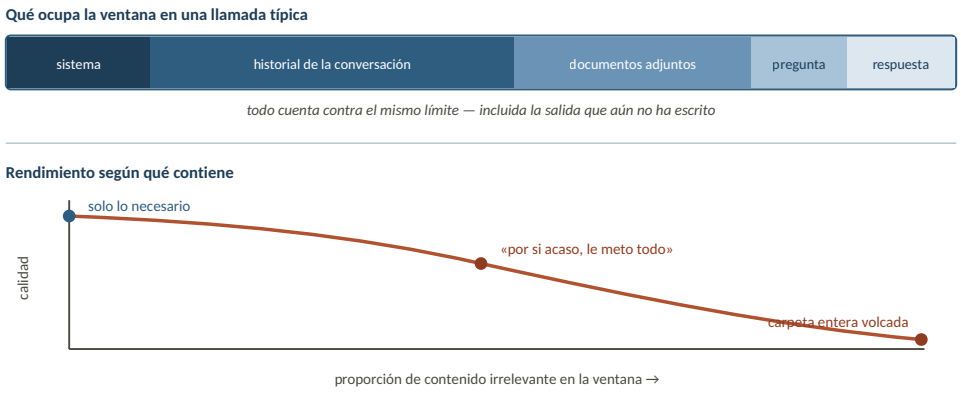


Figura 5.1 · La ventana y la degradación por dilución. Que quepa no significa que convenga: la señal se ahoga en ruido.

Más contexto no es mejor contexto

Este es el hallazgo contraintuitivo que sostiene los volúmenes VIII y IX de esta colección. Un modelo al que se le entrega el documento correcto *junto a otros cuarenta parecidos* responde peor que uno al que se le

entrega solo el correcto. Las ventanas grandes son una comodidad enorme —permiten trabajar con un libro entero, un repositorio, meses de conversación— pero no eximen de decidir qué entra.

Se suma un segundo efecto, bien documentado experimentalmente: **la posición importa**. La información situada al principio y al final de un contexto largo se recupera con más fiabilidad que la que queda enterrada en el medio. Si un dato es crítico, no lo dejes en la página 40 de 80: ponlo al principio, al final, o mejor, no metas 80 páginas.

> ANALOGÍA
>
> Es un escritorio, no un archivador. Cabe mucho encima, pero encontrar el papel que importa se vuelve más difícil según se acumula; y al terminar la jornada alguien barre la mesa entera. Lo que llamamos «memoria» en estas herramientas es un ayudante que, cada mañana, vuelve a poner sobre la mesa lo que anotó de días anteriores.

Consecuencias prácticas

- **Las conversaciones largas se encarecen y se degradan.** Cada turno reenvía todo lo anterior. En trabajos largos, conviene resumir y arrancar limpio en lugar de arrastrar cien mensajes.
- **El contexto es un presupuesto, no un depósito.** Gástalo en lo que cambia la respuesta.
- **La «memoria» es una funcionalidad construida encima**, no una propiedad del modelo: alguien guarda notas fuera y las reinyecta. Por eso puede tener lagunas o requerir que le recuerdes cosas.
- **La respuesta también ocupa.** Pedir un informe muy largo sobre un contexto ya lleno puede truncarlo.

PARA RECORDAR

La ventana de contexto es todo lo que el modelo ve en esta pasada, y no persiste entre llamadas: la conversación se reenvía completa cada vez. Más contexto no es mejor: la dilución degrada la calidad y la posición afecta la recuperación. Trata el contexto como presupuesto y pon lo crítico al principio o al final.

CAPÍTULO 6

Temperatura, muestreo y determinismo

> *La temperatura no controla la creatividad. Controla cuánta probabilidad estás dispuesto a sacrificar.*

El capítulo 1 dejó al modelo con una distribución de probabilidad sobre el siguiente token. Falta elegir uno, y ahí hay decisiones que cambian radicalmente el comportamiento observable.

La opción más simple es tomar siempre el más probable. Produce texto correcto y, en textos largos, curiosamente pobre: repetitivo, plano, propenso a caer en bucles. El lenguaje humano no toma siempre la opción más probable, y un texto que lo hace suena a texto que lo hace. Por eso se *muestrea*: se elige al azar respetando las probabilidades.

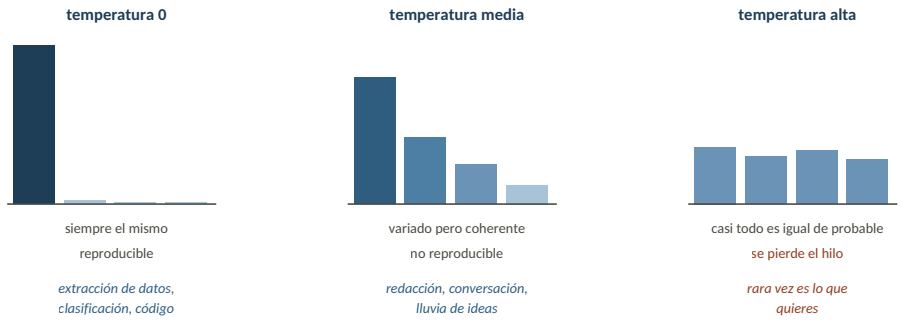


Figura 6.1 · Efecto de la temperatura sobre la distribución. Subirla no añade ideas: aplana las diferencias entre lo probable y lo improbable.

DEFINICIONES

Temperatura: parámetro que aplana o agudiza la distribución antes de muestrear. Baja, concentra la probabilidad en los candidatos más fuertes; alta, la reparte.

Top-p (muestreo por núcleo): restringe la elección al conjunto más pequeño de candidatos cuya probabilidad acumulada alcanza un umbral. Descarta la cola larga de opciones absurdas sin aplanar el resto.

Top-k: restringe la elección a los *k* candidatos más probables. Más rígido que top-p, porque no se adapta a lo segura que esté la distribución.

El mito de la creatividad

La confusión es comprensible pero cuesta cara: subir la temperatura no hace al modelo más creativo, lo hace más *aleatorio*. Una idea original es una conexión inesperada pero pertinente; el ruido no es pertinente. En la práctica, la creatividad se obtiene mucho mejor con el prompt —pidiendo ángulos poco obvios, imponiendo restricciones, solicitando varias alternativas contrastadas— que con el parámetro. La temperatura media existente por defecto suele ser adecuada para escribir.

Cuándo bajar a cero

Hay tareas donde la variabilidad es un defecto puro, y ahí la temperatura mínima es la elección correcta: extracción de campos de un documento, clasificación en categorías fijas, transformación de formatos, generación de código donde quieres reproducibilidad. Es exactamente el caso del capítulo 12 del volumen IX: si el mismo insumo debe producir el mismo resultado, no lo dejes al azar.

MATIZ IMPORTANTE

Temperatura cero *reduce* la variabilidad, pero no garantiza reproducibilidad bit a bit: hay fuentes de no determinismo en el cómputo mismo, y el proveedor puede actualizar el modelo detrás del mismo nombre. Si necesitas garantía real de reproducibilidad, la respuesta es guardar la salida, no confiar en que se pueda regenerar idéntica.

PARA RECORDAR

El modelo produce una distribución y hay que elegir: siempre el más probable produce texto plano y repetitivo, por eso se muestrea. Temperatura aplana o agudiza; top-p recorta la cola absurda. Subir la temperatura da aleatoriedad, no creatividad —esa se pide en el prompt—. Baja a cero para extracción, clasificación y código, sabiendo que reduce la variabilidad sin garantizarla del todo.

CAPÍTULO 7

Por qué alucina

> *No es un error del sistema. Es el sistema funcionando exactamente como fue diseñado, en un caso donde eso no sirve.*

El término es desafortunado porque sugiere una avería. Lo que ocurre es más simple y más inquietante: el modelo produce siempre la continuación más plausible dado el contexto. Cuando el dato correcto está bien representado en lo que aprendió, lo plausible coincide con lo verdadero. Cuando no lo está, **sigue produciendo lo plausible** —porque es lo único que sabe hacer— y lo plausible se parece muchísimo a lo verdadero: tiene el formato correcto, la estructura correcta y el tono correcto.

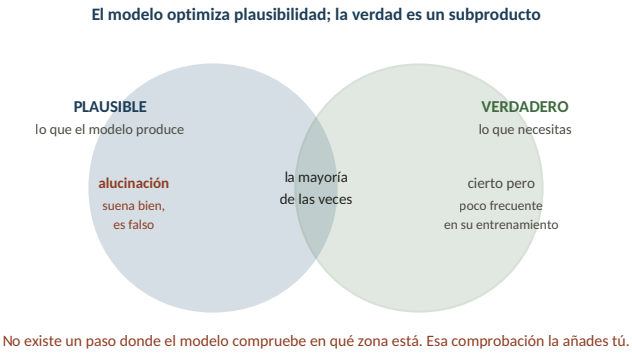


Figura 7.1 · Plausible y verdadero se superponen mucho, pero no coinciden. El modelo no distingue en qué región está operando.

Los cuatro disparadores

Situación	Por qué dispara invención
Dato específico y poco frecuente (una cita, un número de norma, una página)	La forma del dato está bien aprendida; el contenido concreto, no. El resultado tiene formato perfecto y contenido inventado. Es el caso más peligroso.
Premisa falsa en la pregunta («explícame el artículo 47 de la ley X», que no existe)	La continuación más plausible de una pregunta es una respuesta, no una objeción. Hay que pedir explícitamente que cuestione premisas.
Vacío en el contexto (el documento no contiene lo que se pregunta)	Sin instrucción de abstenerse, rellena el hueco con lo que encajaría ahí. Por eso los prompts de los volúmenes anteriores insisten tanto en «si no está, dilo».
Presión conversacional («¿estás seguro?», insistencia, corrección falsa)	El texto que sigue a una objeción suele ser una rectificación. El modelo puede ceder ante una corrección equivocada por pura dinámica textual.

Qué funciona y qué no

Lo que **no** funciona: pedirle que no alucine. «No inventes» ayuda algo en el margen, pero no cambia el mecanismo. Tampoco funciona preguntarle si está seguro: su expresión de confianza es otra generación de texto plausible, no una medición interna.

Lo que sí funciona, en orden de eficacia:

- **Darle el material.** Un modelo que responde *desde* un documento presente en el contexto alucina muchísimo menos que uno que responde de memoria. Es el fundamento del capítulo 10.
- **Exigir citación verificable.** Que cite la ruta, la sección o la línea de donde sale cada afirmación. No elimina el error, pero lo vuelve barato de detectar —que es lo decisivo.
- **Autorizar explícitamente la abstención.** «Si el documento no lo dice, responde: no está en el documento.» Suena trivial y es la mitigación con mejor relación coste-beneficio que existe.

- **Verificación externa.** Para datos críticos, una herramienta que compruebe: buscar la norma, ejecutar el código, consultar la base. El modelo propone, el sistema verifica.

ANALOGÍA

Un guía turístico con enorme experiencia al que le preguntan por un detalle que no conoce. Si su oficio es no dejar preguntas sin respuesta, dirá algo coherente con todo lo demás que sabe, en el mismo tono seguro con el que dijo veinte cosas ciertas. El problema no es su honestidad: es que nadie le autorizó a decir «no lo sé» y nadie está comprobando.

PARA RECORDAR

La alucinación es el sistema haciendo lo suyo —producir lo plausible— donde lo plausible no coincide con lo verdadero. Cuatro disparadores: dato específico poco frecuente, premisa falsa, vacío en el contexto y presión conversacional. Preguntarle si está seguro no sirve. Sirve: darle el material, exigir citación verificable, autorizar la abstención y verificar por fuera lo crítico.

CAPÍTULO 8

Razonamiento extendido

> *Si el modelo construye la respuesta token a token, dejarle escribir más tokens antes de responder cambia lo que puede resolver.*

Vuelve al capítulo 1: cada token se genera viendo todo lo anterior, y cada uno tiene una cantidad limitada de cómputo detrás. De ahí se sigue algo importante: **un problema que requiere varios pasos no cabe en un solo token**. Si el modelo tiene que emitir la respuesta inmediatamente, debe llegar a ella de un salto intuitivo. Si puede escribir el desarrollo primero, cada paso intermedio se convierte en contexto para el siguiente.

Esa es toda la idea. Los modos de razonamiento extendido —con distintos nombres comerciales— son versiones entrenadas y refinadas de esto: el modelo produce un desarrollo interno antes de la respuesta final.

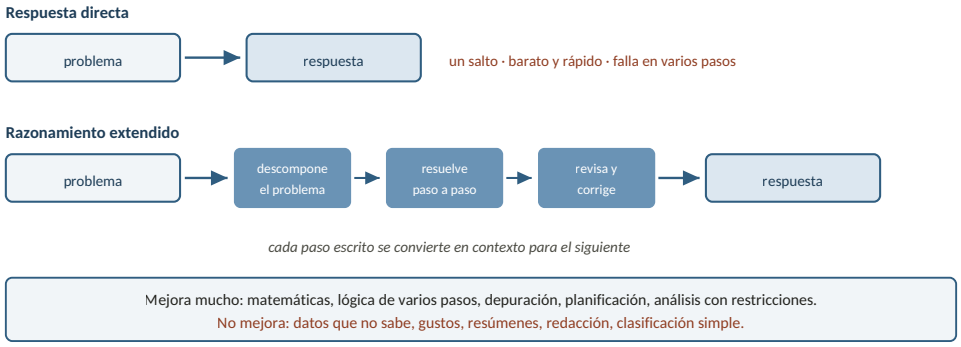


Figura 8.1 · Directo frente a extendido. Los pasos intermedios no son decoración: son el mecanismo.

Qué no arregla

Conviene ser preciso aquí, porque la tentación de activarlo siempre es fuerte. Razonar más no crea información que el modelo no tiene: si desconoce un dato, pensarlo largo rato produce una invención más ela-

borada, no un dato correcto. Tampoco ayuda en tareas de una sola pasada —clasificar un correo, extraer un campo, traducir una frase—, donde solo añade coste y latencia.

Y hay un matiz honesto que merece mención: el desarrollo que produce el modelo es una explicación plausible de un razonamiento, no necesariamente una transcripción fiel del proceso que lo llevó a la respuesta. Es útil —mejora los resultados de forma medible— y a la vez no debe leerse como una auditoría de su funcionamiento interno.

ANALOGÍA

Es la diferencia entre resolver una multiplicación de tres cifras de cabeza o con papel. El papel no te hace más listo: te permite descargar resultados parciales y trabajar sobre ellos. Y para responder «¿cuál es la capital de Francia?» el papel no aporta nada.

PARA RECORDAR

Escribir el desarrollo antes de responder da al modelo pasos intermedios sobre los que apoyarse, y mejora de forma clara matemáticas, lógica de varios pasos, depuración y planificación. No aporta en tareas de una pasada ni crea conocimiento que no tiene. Cuesta más tokens y más latencia: actívalo por tipo de tarea, no por defecto.

PARTE III

Cómo se usa bien

De entender la máquina a obtener resultados fiables: el prompt como especificación, recuperación, herramientas y —lo que separa un experimento de un sistema— evaluación.

CAPÍTULO 9

El prompt como especificación

> *Escribir un buen prompt se parece menos a formular un deseo y más a redactar el encargo que le darías a alguien competente que acaba de llegar.*

Todo lo visto hasta aquí converge en una consecuencia práctica: el prompt no es una orden mágica, es *el contexto que condiciona la predicción*. Un prompt vago no produce un modelo confundido: produce la continuación más plausible de un encargo vago, que es un resultado genérico. La mayor parte de las mejoras vienen de especificar mejor, no de encontrar la palabra secreta.

Los cinco componentes

Componente	Para qué sirve	Ejemplo débil → fuerte
Rol y contexto	Sitúa el registro, el vocabulario y las suposiciones.	«Eres útil» → «Analista de TI escribiendo para gerencia sin formación técnica.»
Tarea	Qué hay que producir, en un verbo concreto.	«Habla del informe» → «Resume los tres hallazgos y su impacto económico.»
Material	Los datos sobre los que trabajar, delimitados.	De memoria → documento adjunto entre delimitadores claros.
Restricciones	Longitud, tono, qué evitar, qué hacer si falta algo.	«Corto» → «Máximo 200 palabras; si el dato no está, dilo.»
Formato	La estructura exacta de salida.	«Ordenado» → «Tabla con columnas: hallazgo, impacto, acción.»

La restricción sobre *qué hacer si falta algo* merece atención especial: es la mitigación del capítulo 7 incorporada al encargo, y es la que más errores evita por palabra escrita.

Ejemplos que enseñan

Dar dos o tres ejemplos de entrada y salida deseada suele funcionar mejor que describir en abstracto lo que quieres, sobre todo cuando el formato es peculiar o el criterio es difícil de verbalizar. Dos advertencias: los ejemplos deben cubrir también los casos raros —si todos son fáciles, el modelo asumirá que todos lo son— y su formato se copia con fidelidad, incluidos los defectos.

```
# Débil
Analiza estos tickets y dime qué pasa.

# Fuerte
Eres analista de mesa de ayuda. Te paso 40 tickets del último mes.

Tarea: identifica los 3 problemas recurrentes que más tiempo consumen.

Para cada uno:
- síntoma tal como lo describe el usuario
- causa raíz probable (marca [hipótesis] si no es concluyente)
- cuántos tickets del lote encajan
- acción sugerida

Restricciones:
- Máximo 150 palabras por problema.
- Si un ticket es ambiguo, cuéntalo aparte como "sin clasificar"; no lo
fuerces.
- No propongas comprar herramientas.

Formato: tabla + un párrafo final de recomendación.

Tickets:
<<<
[contenido]
>>>
```

ERRORES FRECUENTES

Negaciones sin alternativa. «No seas técnico» deja el hueco abierto; «explícalo con una analogía cotidiana» lo llena. **Instrucciones contradictorias** («exhaustivo pero breve») fuerzan al modelo a elegir por ti. **Enterrar la instrucción** al final de veinte páginas de contexto —recuerda el efecto de posición del capítulo 5—. Y **pedir varias tareas distintas en un solo encargo**: casi siempre sale mejor encadenar dos llamadas simples que una compleja.

Iterar con método

La forma habitual de mejorar un prompt —cambiar cosas hasta que «se vea bien»— falla por lo mismo que falla evaluar a ojo (capítulo 12): con un ejemplo, cualquier versión parece buena. El método que funciona es aburrido: reúne cinco o diez casos representativos, incluidos los difíciles; prueba la versión actual contra todos; cambia *una* cosa; vuelve a pasar los mismos casos. Sin eso, no estás iterando, estás paseando.

PARA RECORDAR

El prompt es el contexto que condiciona la predicción, no un conjuro. Cinco componentes: rol, tarea, material, restricciones y formato — e incluye siempre qué hacer si falta información. Los ejemplos enseñan mejor que las descripciones, y deben incluir casos difíciles. Itera cambiando una cosa contra un conjunto fijo de casos.

CAPÍTULO 10

Recuperación aumentada, y cuándo no

> *RAG resolvió un problema real y se convirtió en la respuesta por defecto a preguntas que no siempre lo eran.*

La idea es directa y buena: en lugar de esperar que el modelo sepa algo, se busca el fragmento relevante en un corpus propio y se le entrega en el contexto. Como vimos en el capítulo 7, un modelo que responde desde material presente alucina mucho menos. De ahí su éxito.

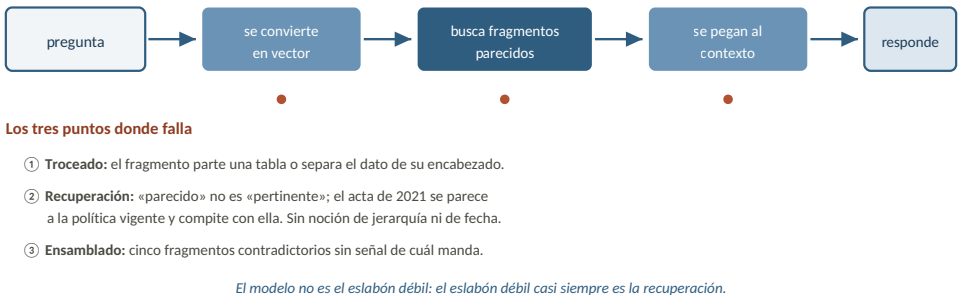


Figura 10.1 · El flujo y sus puntos de fallo. Cuando un sistema RAG responde mal, la causa raíz rara vez está en el modelo.

Cuándo no es la respuesta

Montar recuperación semántica tiene un coste no trivial: trocear, vectorizar, mantener un índice sincronizado, evaluar la calidad de la recuperación. Antes de asumirlo, comprueba si alguna de estas alternativas más baratas resuelve tu caso:

- **Cabe en el contexto.** Si tu corpus son cuarenta procedimientos, quizá quepan enteros. Menos partes móviles, cero fallos de recuperación.
- **Navegación estructurada.** Si tu conocimiento tiene jerarquía y vigencia —y casi todo el conocimiento corporativo la tiene—, un

índice que el agente recorre es más preciso que la similitud vectorial, porque respeta prioridad y fechas. Es la tesis del volumen VIII.

- **Búsqueda literal.** Cuando la gente busca por código, número de contrato o identificador, una búsqueda exacta gana a la semántica sin discusión.
- **Consulta directa a la fuente.** Si el dato está en una base de datos, generar la consulta y ejecutarla da un resultado exacto y verificable en lugar de aproximado.

La combinación que mejor funciona en la práctica no es elegir una, sino **estructura primero, recuperación semántica dentro**: el agente navega el índice hasta el dominio correcto y solo entonces hace búsqueda semántica en ese subconjunto. La jerarquía aporta lo que a la similitud le falta —vigencia y autoridad— y la similitud aporta lo que a la jerarquía le falta: tolerancia al vocabulario.

PARA RECORDAR

RAG funciona porque responder desde material presente reduce la alucinación. Sus tres fallos están en el troceado, la recuperación —«parecido» no es «pertinente», y no distingue vigencia— y el ensamblado de fragmentos contradictorios. Antes de montarlo, comprueba si cabe en el contexto, si la navegación estructurada basta, si la búsqueda literal es mejor o si conviene consultar la fuente. Lo óptimo suele ser estructura primero y semántica dentro.

CAPÍTULO 11

Herramientas, agentes y el bucle

> *Un agente no es un modelo más inteligente. Es el mismo modelo dentro de un bucle que le permite mirar el resultado de sus actos.*

Un modelo produce texto. La transición a agente ocurre cuando ese texto puede ser una *llamada a una herramienta*: buscar en la web, leer un archivo, ejecutar código, consultar una base. El sistema que rodea al modelo ejecuta la llamada, devuelve el resultado al contexto, y el modelo continúa. Eso es todo el mecanismo, y explica tanto lo que los agentes hacen bien como dónde se pierden.

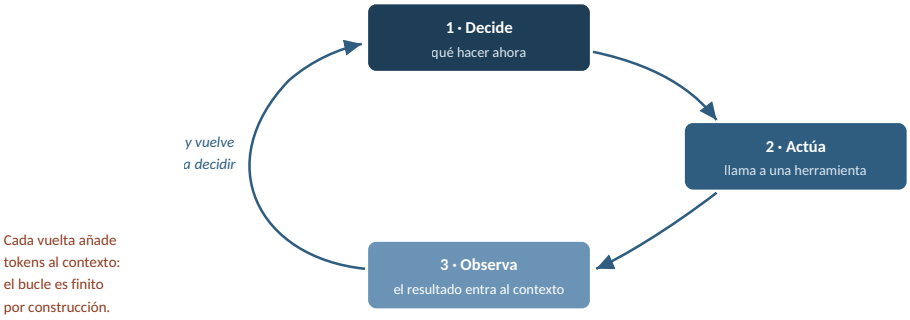


Figura 11.1 · El bucle. La capacidad nueva no es cognitiva: es poder ver el resultado de un acto antes de decidir el siguiente.

Qué gana el sistema

Tres cosas, y las tres compensan limitaciones vistas en capítulos anteriores. **Datos que no tiene**: cualquier cosa posterior al corte de conocimiento o privada de tu organización. **Exactitud**: en lugar de estimar un cálculo, lo ejecuta —recuerda el capítulo 2 y la aritmética—. **Corrección sobre la marcha**: si el código falla, ve el error y lo arregla, lo cual convierte un fallo silencioso en un ciclo de mejora.

Dónde se pierden

Modo de fallo	Por qué y qué hacer
Acumulación de contexto	Cada observación se suma. A las quince vueltas, el contexto está lleno de resultados intermedios y aparece la dilución del capítulo 5. Mitigación: resumir estados intermedios, no arrastrarlo todo.
Bucles improductivos	Reintenta la misma acción esperando un resultado distinto. Mitigación: límite de vueltas y de reintentos por herramienta, con salida declarada.
Deriva del objetivo	Tras muchos pasos, resuelve un problema adyacente al planteado. Mitigación: reafirmar el objetivo en cada vuelta o dividir la tarea.
Confianza en la herramienta	Si la herramienta devuelve algo incorrecto, el modelo lo integra sin cuestionarlo. La calidad del agente está acotada por la de sus herramientas.
Acciones irreversibles	El bucle corrige lo que puede reintentar. Enviar un correo o borrar un registro no se deshace. Mitigación: confirmación humana para lo irreversible.

ANALOGÍA

Es la diferencia entre responder un examen a libro cerrado y trabajar en un taller. En el taller puedes medir, probar, ver que no encaja y rectificar. No te vuelve mejor artesano; te da realimentación. Y con malas herramientas de medición, el taller no salva la pieza.

PARA RECORDAR

Un agente es el modelo dentro de un bucle decidir–actuar–observar donde el resultado de cada acción entra al contexto. Aporta datos que no tiene, exactitud por ejecución y corrección sobre la marcha. Se pierde por acumulación de contexto, bucles improductivos, deriva del objetivo y confianza ciega en sus herramientas. Lo irreversible requiere confirmación humana.

CAPÍTULO 12

Evaluación: medir en vez de opinar

> *Probaste cinco ejemplos, salieron bien y concluiste que funciona. Cinco ejemplos no distinguen un 95 % de acierto de un 70 %, y esa diferencia es todo.*

Esta es la disciplina que separa un experimento simpático de un sistema en el que se puede confiar, y la que más se omite. Hay dos motivos por los que la impresión engaña de forma sistemática. Primero, la salida siempre *suena* bien: la fluidez es constante, la corrección no. Segundo, quien evalúa suele ser quien escribió el prompt, y prueba inconscientemente los casos que tenía en mente al escribirlo.

El set de evaluación

Como en los volúmenes VIII y IX: entre veinte y cincuenta casos reales, con la salida correcta verificada por una persona, guardados en un archivo. Se ejecutan cada vez que cambias algo. Tres reglas que hacen la diferencia:

- **Casos reales, no inventados.** Los inventados siempre son más limpios que lo que llega en producción.
- **Incluye los difíciles y los imposibles.** Preguntas ambiguas, con premisa falsa, o cuya respuesta correcta es «no está en el material». Sin ellas mides solo el camino feliz —y los fallos caros del capítulo 7 viven justo ahí.
- **Congélalo.** Si ajustas el set cada vez que falla algo, dejas de poder comparar entre versiones. Se amplía deliberadamente, no en caliente.

Cómo se puntúa lo que no tiene respuesta única

Tipo de tarea	Método	Nota
Clasificación, extracción	Comparación exacta	Automático y barato. Empieza por convertir lo que puedas a esta forma.
Respuesta con dato correcto	Verificación del dato clave	No compares el texto entero: comprueba si contiene el hecho.
Redacción, resumen	Rúbrica de 3–5 criterios	Cubre lo esencial, no inventa, longitud, tono. Puntuar de 0 a 2 basta.
Volumen alto	Modelo como juez con rúbrica	Útil y con sesgos propios: calibra contra un lote puntuado a mano.

SOBRE EL MODELO COMO JUEZ

Es una técnica práctica y a estas alturas habitual, pero tiene sesgos documentados: tiende a favorecer respuestas largas, y a puntuar mejor las que se parecen a su propio estilo. Sirve para detectar regresiones y para ordenar alternativas; no sirve como medida absoluta de calidad ni como única evidencia para poner algo en producción.

Regresiones

El motivo más práctico de todos para tener un set: los sistemas con modelos *retroceden* de formas que el software tradicional no. Cambias un prompt para arreglar un caso y rompes tres. Actualizas a un modelo mejor y una tarea concreta empeora. El proveedor actualiza detrás del mismo nombre y algo se mueve. Sin un set fijo, te enteras por una queja; con él, en cinco minutos.

PARA RECORDAR

La impresión engaña porque la fluidez es constante y la corrección no. Veinte a cincuenta casos reales con respuesta verificada, incluidos ambiguos y los que deben responderse «no está»; congelado para poder comparar. Comparación exacta donde se pueda, rúbrica corta donde no, y el modelo como juez solo para detectar regresiones. Es lo único que convierte «creo que mejoró» en un número.

PARTE IV

Decidir con criterio

Dos capítulos finales: cómo elegir un modelo sin quedar atrapado en comparativas que caducan, y qué límites conviene asumir antes de construir encima.

CAPÍTULO 13

Elegir modelo

> *La pregunta «¿cuál es el mejor?» tiene la misma utilidad que «¿cuál es el mejor vehículo?». La respuesta empieza por qué vas a transportar.*

Cualquier comparativa concreta que escriba aquí estará desactualizada antes de que la leas. Lo que no caduca son los ejes de decisión, que llevan años siendo los mismos tres y suelen estar en tensión.

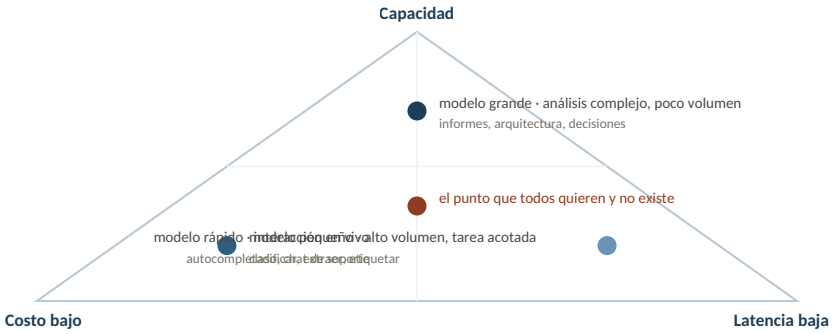


Figura 13.1 · Los tres ejes en tensión. Elegir es decidir qué sacrificar, y la mezcla suele batir a cualquier elección única.

La mezcla de modelos

La estrategia que mejor funciona en producción es tan poco glamorosa como eficaz: rutear por dificultad. El modelo pequeño y barato atiende el grueso de las tareas acotadas —clasificar, extraer, formatear— y el grande entra solo donde hace falta capacidad. En sistemas con volumen real, la diferencia de coste entre hacer esto y no hacerlo es de un orden de magnitud, sin pérdida de calidad perceptible.

El corolario, que conviene tener presente desde el primer día: **diseña para poder cambiar de modelo**. Los prompts, el set de evaluación y la lógica del sistema deben ser independientes del proveedor. Si cam-

biar de modelo requiere reescribirlo todo, quedas atado a decisiones que caducan cada pocos meses.

Local frente a servicio

Consideración	Cómo suele resolverse
Datos que no pueden salir	Es el argumento fuerte a favor de local. Pero antes verifica qué garantías contractuales ofrece el proveedor: a menudo cubren el requisito real.
Costo a gran volumen	Local puede salir a cuenta con volumen alto y sostenido; con volumen irregular, casi nunca, porque el hardware no se apaga.
Capacidad	Los modelos abiertos que caben en hardware razonable van por detrás de los mayores servicios en tareas difíciles. La distancia se acorta y vuelve a abrirse; no la des por fija.
Operación	El coste oculto: servir un modelo en producción es infraestructura con guardias, no un proyecto de fin de semana.

PARA RECORDAR

Tres ejes en tensión —capacidad, costo, latencia— y ninguna elección los maximiza. La estrategia ganadora es la mezcla: modelo barato para el volumen acotado, modelo capaz para lo difícil. Diseña para poder cambiar de modelo: prompts, evaluación y lógica independientes del proveedor. Local se justifica por restricciones de datos o volumen alto sostenido, contando la operación como parte del coste.

CAPÍTULO 14

Límites, riesgos y qué esperar

> *La pregunta interesante no es qué no pueden hacer hoy. Es cuáles de esas limitaciones son de tamaño y cuáles son de naturaleza.*

Es un buen cierre porque obliga a ser honesto sobre lo que sabemos y lo que no. Algunas limitaciones se han ido resolviendo con modelos mejores; otras llevan años ahí y hay razones estructurales para pensar que seguirán, al menos con esta arquitectura.

Lo que no resuelve escalar

- **El conocimiento tiene fecha.** Un modelo entrenado no se actualiza solo. Esto no se arregla haciéndolo más grande; se arregla con herramientas y contexto.
- **No sabe lo que no sabe.** No hay una medida interna de confianza que puedas consultar. Su seguridad expresada es texto, no un dato. Por eso la verificación externa es arquitectura, no desconfianza.
- **La aritmética exacta y la manipulación de caracteres** chocan con la tokenización del capítulo 2. Se mitigan con herramientas, no con escala.
- **La opacidad.** Nadie puede explicar del todo por qué un modelo produjo una salida concreta. La interpretabilidad avanza, pero hoy, para un sistema que debe justificar decisiones, la trazabilidad la aporta tu diseño —citas, fuentes, registros— no el modelo.

Riesgos que exigen decisión, no vigilancia

Sesgo. El modelo aprendió de texto humano y arrastra sus regularidades, incluidas las indeseables. En tareas que afectan a personas —selección, evaluación, priorización— esto no es un detalle técnico: es la razón por la que esas decisiones necesitan revisión humana y criterios explícitos, no solo una buena instrucción.

Confianza mal calibrada. Funciona bien treinta veces y dejas de verificar la trigésimo primera. No se corrige con advertencias: se corrige haciendo que verificar cueste un clic —citas exactas y abribles— y reservando la verificación obligatoria para lo que tiene consecuencias.

Atrofia. Si el sistema responde siempre, el equipo deja de construir criterio propio. Se nota el día que hay que decidir bajo presión sin la herramienta. La mitigación de diseño es la misma del volumen VIII: pedir que explique el camino, no solo el resultado.

Cómo envejecerá este libro

Con franqueza: los capítulos 1 al 3 y el 5 al 7 describen mecanismos que llevan años estables y probablemente sigan siendo correctos durante bastante tiempo. Los capítulos 8, 10 y 13 son los más expuestos: el razonamiento extendido, las técnicas de recuperación y el panorama de modelos se mueven rápido, y algunas de sus recomendaciones prácticas envejecerán antes.

Lo que no espero que cambie es la disposición de fondo. Una máquina que produce lenguaje plausible con enorme competencia y sin acceso a la verdad exige un diseño con verificación, límites claros y criterio humano en los puntos que importan. Todo lo demás de este libro son consecuencias de eso.

Y volviendo a la portada: seguimos delante de un cerebro extraordinario del que medimos el comportamiento mejor de lo que explicamos el mecanismo. Trabajar bien con él consiste, en buena medida, en no confundir una cosa con la otra.

PARA RECORDAR

No se arreglan con escala: la fecha del conocimiento, la ausencia de confianza calibrada, la aritmética exacta y la opacidad. Tres riesgos exigen decisión de diseño: sesgo en decisiones sobre personas, confianza mal calibrada y atrofia del criterio propio. Los mecanismos de la Parte I envejecen bien; las recomendaciones prácticas, menos. La disposición que no caduca: verificación, límites y criterio humano donde importa.

APÉNDICE A

Glosario de terminología

Agente

Modelo operando dentro de un bucle decidir–actuar–observar, donde el resultado de cada llamada a herramienta se incorpora al contexto antes de la siguiente decisión.

Alineación

Tercera etapa de entrenamiento, en la que el modelo aprende a preferir respuestas útiles, honestas y seguras a partir de señales de preferencia. Aporta la mayor parte del carácter observable.

Alucinación

Producción de contenido plausible pero falso. No es una avería: es el mecanismo de predicción operando donde lo plausible no coincide con lo verdadero.

Atención

Mecanismo que permite a cada posición del texto atender directamente a cualquier otra, mediante consultas, claves y valores evaluados en paralelo. Su coste crece de forma cuadrática con la longitud.

Autorregresivo

Propiedad de un modelo cuya propia salida se convierte en parte de su siguiente entrada, generando token a token.

Corte de conocimiento

Fecha hasta la que llega el texto usado en el preentrenamiento. Todo lo posterior requiere contexto o herramientas; lo cercano al corte suele estar peor representado.

Degradación por dilución

Caída de calidad cuando el material relevante llega acompañado de mucho contenido irrelevante en la misma ventana.

Embedding

Representación vectorial de un token o un fragmento, en la que la proximidad geométrica corresponde a similitud de uso y significado.

Modelo como juez

Técnica de evaluación en la que un modelo puntúa salidas según una rúbrica. Útil para detectar regresiones; tiene sesgos propios y no sirve como medida absoluta.

Preentrenamiento

Primera etapa: predecir el siguiente token sobre grandes volúmenes de texto. Concentra la mayor parte del cómputo y determina qué sabe el modelo.

Prompt

Texto que condiciona la predicción. Funciona como especificación de un encargo: rol, tarea, material, restricciones y formato.

RAG (recuperación aumentada)

Patrón que busca fragmentos relevantes en un corpus propio y los inserta en el contexto antes de responder. Sus fallos suelen estar en el troceado y la recuperación, no en el modelo.

Razonamiento extendido

Modo en el que el modelo produce un desarrollo antes de la respuesta final, ganando pasos intermedios sobre los que apoyarse. Mejora tareas de varios pasos; no crea conocimiento que no tiene.

Temperatura

Parámetro que aplana o agudiza la distribución antes de muestrear. Aumenta la aleatoriedad, no la creatividad.

Token

Unidad mínima de texto que procesa el modelo, de tamaño variable. Los límites de contexto y los precios se miden en tokens; el modelo no accede a las letras que hay dentro de uno.

Top-p (muestreo por núcleo)

Restringe la elección al conjunto más pequeño de candidatos cuya probabilidad acumulada alcanza un umbral, descartando la cola larga de opciones absurdas.

Ventana de contexto

Todo el texto que el modelo considera en una pasada: sistema, historial, adjuntos, pregunta y respuesta en curso. No persiste entre llamadas.

COLOFÓN

Sobre esta colección

Décimo volumen de una biblioteca sobre agentes de IA escrita e ilustrada íntegramente desde cero: texto original, diagramas propios y una portada con un animal elegido como metáfora del tema.

#	Título	Animal	Ángulo
I	De la idea a producción	Pulpo	Inteligencia distribuida
II	Construyendo multiagente	Hormiga	Enjambre emergente
III	Guía ilustrada	Cuervo	Uso de herramientas
IV	Patrones arquitectónicos	Abeja	Patrones probados
V	Agentes con MCP	Delfín	Comunicación y protocolo
VI	En acción	Zorro	Acción inteligente
VII	En la práctica	Castor	Ingeniería para producción
VIII	El segundo cerebro corporativo	Ardilla	Conocimiento recuperable
IX	La federación de cerebros	Suricata	Vigilancia distribuida
X	**La máquina de predecir**	**Cachalote**	**El motor por dentro**

Escrito e ilustrado por Claude · Anthropic
Edición de autor · Primera edición · 2026